

Evaluating gpt-oss for Refactoring Detection: A Replication of RefModel

Pedro Simões

Federal University of Campina Grande

Brazil

pedro.henrique.lima.simoess@ccc.ufcg.edu.br

Rohit Gheyi

Federal University of Campina Grande

Brazil

rohit@dsc.ufcg.edu.br

Abstract

Automatic refactoring detection is a fundamental task in software engineering, used to understand system evolution, support code reviews, and enable empirical analyses. The RefModel study [12] demonstrated that foundation models such as Claude 3.5 Sonnet achieve up to 98.5% recall in detecting refactorings in synthetic Java programs, outperforming traditional static-analysis-based tools. However, the study only evaluated a limited set of models and did not include gpt-oss, OpenAI’s first open-weight model [8], available for local execution via Ollama. In this work, we present a replication of the original study replacing the evaluated models with gpt-oss-20b, executed locally via Ollama, using the same methodology, dataset, and metrics. Our results show that gpt-oss-20b achieves 95.4% recall on synthetic programs and 86.4% on real-world programs, surpassing Phi4-14B in both settings and performing competitively with Claude 3.5 Sonnet, while running fully locally at zero API cost.

Keywords

Refactoring Detection, Foundation Models, RefModel, gpt-oss

1 Introduction

Refactoring is the process of restructuring code to improve its readability, maintainability, and design quality without altering its external behavior [3]. Recent studies indicate that more than 40% of developers perform refactorings daily [4], highlighting the relevance of this practice in the software development lifecycle.

Accurate refactoring detection is essential to support tasks such as code review, change integration, program comprehension, and empirical studies on the impact of refactoring on software quality [9]. To automate this task, the research community has developed tools such as RefactoringMiner [14], RefDiff [10], and ReExtractor+ [5], which analyze version histories to identify applied transformations. However, these approaches rely on manually crafted rules and complex static analysis, making them difficult to extend to new refactoring types or programming languages.

With the emergence of foundation models, a promising alternative has been explored: using large language models (LLMs) to detect refactorings through learned representations and natural language prompts [2]. Simões et al. [12] introduced RefModel, evaluating models such as Phi4-14B, Claude 3.5 Sonnet, Gemini 2.5 Pro, and o4-mini-high on synthetic Java programs and real-world systems. Among local models, only Phi4-14B — a general-purpose model — was evaluated, achieving 79.4% recall on small programs and 79.7% on real-world systems.

A significant development since the original study is the release of **gpt-oss** [8], OpenAI’s first open-weight model family since GPT-2. The gpt-oss-20b variant delivers performance comparable to o3-mini on common benchmarks and can run on hardware with as little as 16 GB of memory via Ollama [7]. This makes it a compelling candidate for local refactoring detection: it combines strong reasoning capabilities with full data privacy and zero API cost, addressing practical concerns that limit adoption of cloud-based models in industrial settings.

Goal. This work replicates the experiment of Simões et al. [12] by replacing the evaluated models with gpt-oss-20b, executed locally via Ollama, keeping all other aspects of the original methodology unchanged. We aim to answer the following research questions:

- **RQ1:** To what extent does gpt-oss-20b detect refactorings in small synthetic Java programs?
- **RQ2:** To what extent does gpt-oss-20b detect refactorings in real-world Java programs?
- **RQ3:** How does the performance of gpt-oss-20b compare to the models evaluated in the original study, particularly Phi4-14B and Claude 3.5 Sonnet?

This replication contributes to understanding the viability of open-weight reasoning models for software engineering tasks, and evaluates the feasibility of fully local, cost-free refactoring detection pipelines.

2 Methodology

This work is a replication of the experiment by Simões et al. [12], fully preserving the original methodology and replacing only the evaluated model with gpt-oss-20b, executed locally via Ollama.

2.1 Experiment Design

We adopt the same goal as the original study: to assess the effectiveness of foundation models in detecting refactorings from a developer’s perspective, using transformations applied to both synthetic small Java programs and real-world open-source systems.

The study is comparative and experimental. The experiment variables are:

- **Independent variable:** The language model used for refactoring detection (gpt-oss-20b vs. models from the original study).
- **Dependent variables:** Precision, Recall, and the pass@1 metric [1] per refactoring type.
- **Metrics:** Recall and Precision computed per refactoring type and in aggregate, following exactly the methodology of [12].

2.2 Dataset

We used the same two datasets provided by the RefModel authors [11].

Synthetic programs. The first dataset consists of **858 transformations** applied to Java programs automatically generated by the JDolly tool [6], ranging from 14 to 34 lines of code. Each transformation represents a single refactoring automatically applied via Eclipse JDT 4.16, covering **10 refactoring types**: *Add Parameter*, *Encapsulate Field*, *Move Method*, *Pull Up Field*, *Pull Up Method*, *Push Down Field*, *Push Down Method*, *Rename Field*, *Rename Method*, and *Rename Class*. Incorrect transformations were previously filtered using the Java compiler and the SafeRefactor tool [13].

Real-world programs. The second dataset consists of **44 transformations** taken from real-world open-source Java projects: Lettuce, Apache Gobblin, Google Maps Services, Spring Boot, and RefMiner (see Table 3 in [12]). These transformations cover 12 refactoring types, including *Extract Class*, *Extract Interface*, *Inline Method*, and *Move Method*, among others.

Both datasets are publicly available with no access restrictions.

2.3 Evaluated Model

We evaluated **gpt-oss-20b** [8], an open-weight model released by OpenAI in August 2025, executed locally via the Ollama platform [7]. The results are compared against Tables 2 and 4 of [12], which report results for synthetic and real-world programs respectively.

2.4 Prompt

We used exactly the same prompts from the original study [12], with no modifications. For synthetic programs, the prompt provides the original and transformed program pair. For real-world programs, the prompt provides the GitHub diff of the commit. In both cases, the model uses the same predefined refactoring definitions based on Fowler’s catalog [3].

2.5 Execution Environment

gpt-oss-20b was executed locally via the **Ollama** platform on a MacBook Pro equipped with an M3 processor and 18 GB of RAM. The temperature was set to 0.6, following the configuration adopted for Phi4-14B in the original study. All other parameters were kept at their default values.

2.6 Analysis Procedure

For each transformation, the model received the program pair (or diff) via prompt and identified the refactoring type. Responses were compared against the ground truth and independently assessed.

3 Results and Discussion

3.1 RQ1: Synthetic Programs

Table 1 presents the performance of gpt-oss-20b on the 858 synthetic transformations, compared against Phi4-14B, Claude 3.5 Sonnet, and RefactoringMiner from the original study [12].

gpt-oss-20b achieved a recall of **95.4%** and precision of **94.5%** on the synthetic dataset, correctly identifying 819 out of 858 transformations. This represents a substantial improvement over Phi4-14B (79.4% recall), the only other local model evaluated in the original

Table 1: Performance of each approach on small programs.

Refactoring	Phi4	CLAUDE	RMINER	GPT-oss	Total
Add Parameter	97	100	100	100	100
Encapsulate Field	100	100	100	100	100
Move Method	43	93	100	76	100
Pull Up Field	85	100	100	100	100
Pull Up Method	37	45	45	45	47
Push Down Field	52	100	100	97	100
Push Down Method	0	9	3	10	11
Rename Class	90	99	100	94	100
Rename Field	65	99	100	100	100
Rename Method	99	100	74	97	100
Recall	79.4%	98.5%	95.8%	95.4%	100%
Precision	56.4%	88.5%	89.3%	94.5%	100%

Table 2: Performance on real-world programs.

Refactoring	Phi4	CLAUDE	GEMINI	04-MINI	GPT-oss	RMINER	REFDIFF	REEXT.	Total
Extract Class	100%	0%	0%	0%	100%	100%	100%	100%	1
Extract Interface	100%	100%	75%	100%	100%	100%	100%	75%	8
Extract Superclass	100%	100%	100%	100%	100%	100%	100%	100%	2
Inline Method	80%	100%	100%	100%	80%	60%	40%	60%	5
Move Method	38%	100%	100%	87.5%	100%	75%	100%	87.5%	8
Pull Up Field	100%	100%	100%	100%	50%	100%	0%	100%	2
Pull Up Method	80%	80%	80%	100%	80%	100%	100%	100%	5
Push Down Field	0%	100%	100%	100%	100%	100%	100%	100%	1
Push Down Meth.	50%	75%	100%	100%	50%	75%	50%	75%	4
Rename Class	100%	100%	100%	100%	100%	100%	100%	100%	4
Rename Field	100%	100%	100%	66.6%	100%	100%	100%	100%	1
Rename Method	100%	100%	100%	66.6%	100%	100%	100%	100%	3
Recall	79.7%	92.2%	93.8%	93.8%	86.4%	88.6%	77.3%	84.1%	100%
Precision	61.4%	77.6%	82.2%	81.1%	84.4%	41.9%	70.8%	40.2%	100%

study, and closely approaches the performance of Claude 3.5 Sonnet (98.5%). Notably, gpt-oss-20b also surpasses RefactoringMiner (95.8% recall) in precision, while matching it closely in recall.

3.2 RQ2: Real-World Programs

Table 2 presents the performance of gpt-oss-20b on the 44 real-world transformations.

On real-world programs, gpt-oss-20b achieved a recall of **86.4%** and precision of **84.4%**, correctly identifying 38 out of 44 transformations. This places it between Phi4-14B (79.7%) and Claude 3.5 Sonnet (92.2%), surpassing RefDiff (77.3%) and ReExtractor+ (84.1%), and approaching RefactoringMiner (88.6%). Notably, gpt-oss-20b achieved the highest precision (84.4%) among all approaches evaluated on this dataset.

False positives included incorrect identifications of *Add Method Parameter*, *Move Field*, and *Pull Down Method*, non-standard labels not present in the predefined taxonomy.

3.3 RQ3: Comparison with Original Study

gpt-oss-20b consistently outperforms Phi4-14B, the only other local model evaluated in the original study, in both settings: +16 percentage points in recall on synthetic programs (95.4% vs. 79.4%) and +6.7 points on real-world programs (86.4% vs. 79.7%). In terms of precision, gpt-oss-20b (94.5%) substantially surpasses Phi4-14B (56.4%) on synthetic programs and (84.4% vs. 61.4%) on real-world ones.

When compared to Claude 3.5 Sonnet, gpt-oss-20b performs competitively on synthetic programs (95.4% vs. 98.5% recall) but lags behind on real-world programs (86.4% vs. 92.2%). However, gpt-oss-20b achieves higher precision than Claude 3.5 Sonnet in both settings (94.5% vs. 88.5% on synthetic; 84.4% vs. 77.6% on real-world), generating fewer false positives.

These results confirm that gpt-oss-20b is a viable local alternative to proprietary models for refactoring detection, offering strong performance without API costs or data privacy concerns.

4 Threats to Validity

Internal validity. We used the original RefModel replication package [11], executing gpt-oss-20b through the same scripts, prompts, and pipeline used to evaluate the other models, which eliminates implementation differences as a source of variation. However, slight variations in model behavior due to non-determinism (even at temperature 0.6) may affect individual responses. Following the same pass@1 protocol as the original study, each transformation was executed once; repeated runs could yield slightly different results.

External validity. Our synthetic dataset consists of small programs generated by JDolly (14–34 LOC), which may not fully represent the complexity of real-world code. The real-world dataset, while more realistic, comprises only 44 transformations from five Java projects, which limits the generalizability of our findings to larger and more diverse codebases.

Construct validity. Ambiguities in refactoring taxonomy may affect results. As observed in the false positive analysis, gpt-oss-20b occasionally produced labels outside the predefined list (e.g., *Pull Down Method*, *Move Field*), suggesting sensitivity to prompt phrasing. This threat is shared with the original study.

5 Conclusion

In this paper, we replicated the RefModel experiment [12] by evaluating gpt-oss-20b, OpenAI's first open-weight reasoning model, on refactoring detection using the same datasets, prompts, and metrics as the original study. Our results show that gpt-oss-20b achieves 95.4% recall and 94.5% precision on synthetic Java programs, and 86.4% recall and 84.4% precision on real-world programs. These results substantially surpass Phi4-14B, the only other local model in the original study, and are competitive with Claude 3.5 Sonnet, a proprietary API-based model.

The key contribution of this replication is demonstrating that a fully local, open-weight reasoning model can approach the performance of state-of-the-art proprietary models for refactoring detection, while eliminating API costs and data privacy concerns. This is particularly relevant for industrial adoption, where sending proprietary code to external services may not be acceptable.

Reproducibility

All artifacts of this study are publicly available at <https://github.com/pedrinhoeu/RefModel-gpt-oss-replication>, including the gpt-oss-20b results per refactoring type and instructions to reproduce the experiment. The datasets, prompts, and execution pipeline are those of the original RefModel replication package [11], used without modification; only the model configuration was changed. Reproducing the experiment requires Ollama with the gpt-oss-20b model and the setup described in Section 2.5.

References

- [1] Mark Chen et al. 2021. Evaluating Large Language Models Trained on Code. <https://arxiv.org/abs/2107.03374>
- [2] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *International Conference on Software Engineering: Future of Software Engineering*. IEEE, 31–53.
- [3] Martin Fowler. 1999. *Refactoring: improving the design of existing code*. Addison-Wesley.
- [4] Yaroslav Golubev, Zarina Kurbatova, Eman Abdullah AlOmar, Timofey Bryksin, and Mohamed Wiem Mkaouer. 2021. One thousand and one stories: a large-scale survey of software refactoring. In *Foundations of Software Engineering*. 1303–1313.
- [5] Bo Liu, Hui Liu, Nan Niu, Yuxia Zhang, Guangjie Li, He Jiang, and Yanjie Jiang. 2025. An Automated Approach to Discovering Software Refactorings by Comparing Successive Versions. *IEEE Transactions on Software Engineering* 51, 5 (2025), 1358–1380.
- [6] Melina Mongiovi, Rohit Gheyi, Gustavo Soares, Márcio Ribeiro, Paulo Borba, and Leopoldo Teixeira. 2018. Detecting Overly Strong Preconditions in Refactoring Engines. *IEEE Transactions on Software Engineering* 44, 5 (2018), 429–452.
- [7] Ollama. 2025. OpenAI gpt-oss. <https://ollama.com/library/gpt-oss>.
- [8] OpenAI. 2025. Introducing gpt-oss. <https://openai.com/index/introducing-gpt-oss/>.
- [9] Fabio Palomba, Andy Zaidman, Rocco Oliveto, and Andrea De Lucia. 2017. An Exploratory Study on the Relationship between Changes and Refactoring. In *International Conference on Program Comprehension*. 176–185.
- [10] Danilo Silva, João Paulo da Silva, Gustavo Santos, Ricardo Terra, and Marco Tulio Valente. 2021. RefDiff 2.0: A Multi-language Refactoring Detection Tool. *IEEE Transactions on Software Engineering* 47, 12 (2021), 2786–2802.
- [11] P. Simões, R. Gheyi, R. Melo, J. Oliveira, M. Ribeiro, and W. Assunção. 2025. REFMODEL: Detecting Refactorings using Foundation Models (artifacts). <https://github.com/brain-ufcg/RefModel/>.
- [12] Pedro Simões, Rohit Gheyi, Rian Melo, Jonhnathan Oliveira, Márcio Ribeiro, and Wesley K. G. Assunção. 2025. RefModel: Detecting Refactorings using Foundation Models. In *Brazilian Symposium on Software Engineering (SBES)* (Recife, PE, Brasil).
- [13] Gustavo Soares, Rohit Gheyi, Dalton Serey, and Tiago Massoni. 2010. Making Program Refactoring Safer. *IEEE Software* 27, 4 (2010), 52–57.
- [14] Nikolaos Tsantalis, Ameya Ketkar, and Danny Dig. 2022. RefactoringMiner 2.0. *IEEE Transactions on Software Engineering* 48, 3 (2022), 930–950.